

Exceptional and File Handling

1. What Is an Exception?

An **exception** is an **unexpected event** that occurs during program execution and disrupts the normal flow of instructions.

Common examples

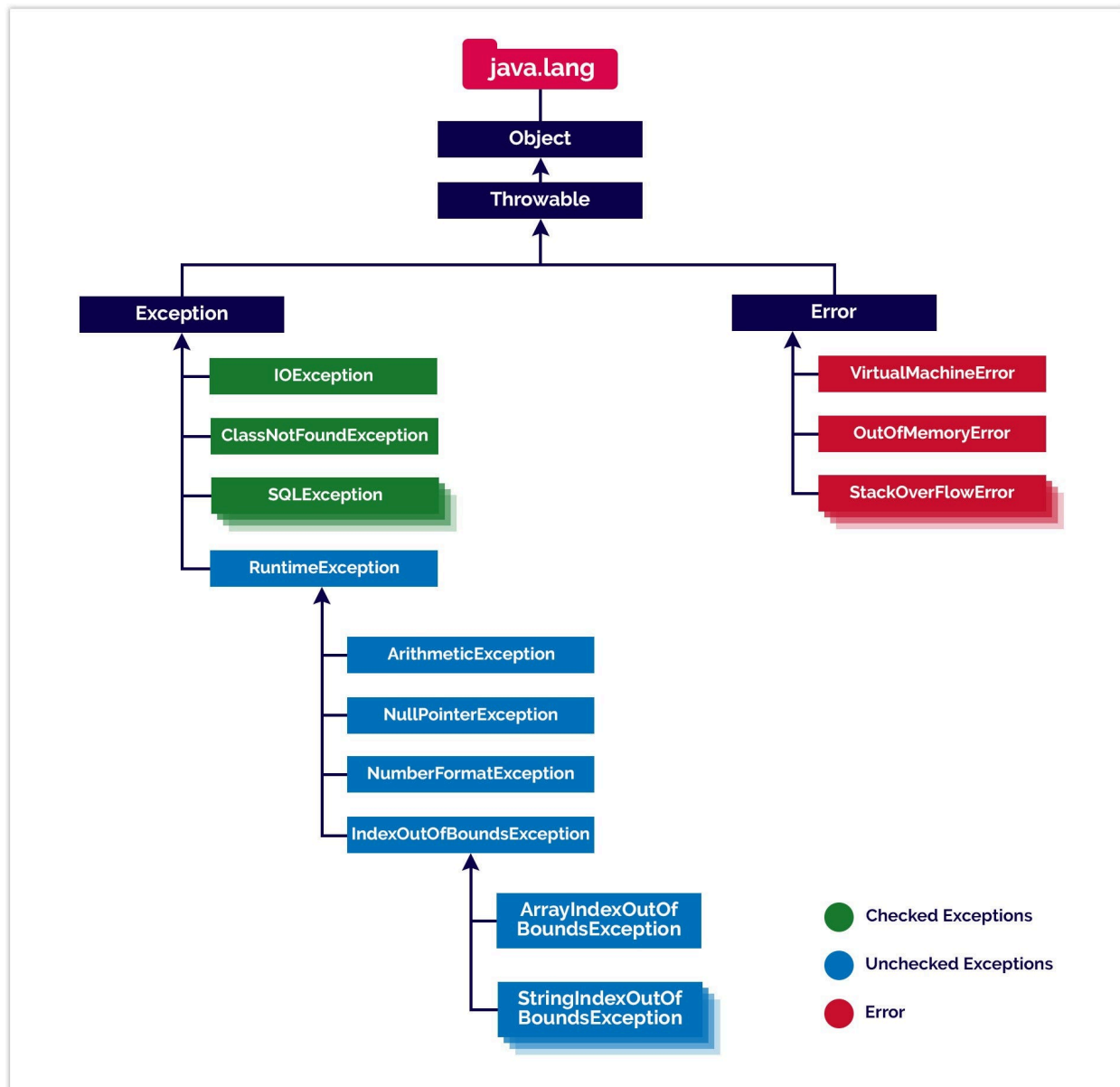
- Dividing a number by zero
 - Accessing an invalid array index
 - Opening a file that does not exist
 - Null object access
-

2. Why Exception Handling Is Important

- Prevents program crashes
 - Separates **error-handling logic** from normal logic
 - Improves code readability and maintainability
 - Enables recovery from errors
-

3. Exception Hierarchy in Java

Java exceptions are organized in a hierarchy rooted at the `Throwable` class.



Main branches

1. Throwable

- Error
- Exception

3.1 Error

- Serious problems that applications **should not try to handle**

- Example: `OutOfMemoryError` , `StackOverflowError`

3.2 Exception

- Conditions that applications **can handle**
 - Divided into:
 - **Checked Exceptions**
 - **Unchecked Exceptions**
-

4. Checked vs Unchecked Exceptions

4.1 Checked Exceptions

- Checked at **compile time**
- Must be handled using `try-catch` or declared using `throws`
- Examples:
 - `IOException`
 - `SQLException`
 - `ClassNotFoundException`

```
FileReader fr = new FileReader("file.txt");// Compile-time error if not handled
```

4.2 Unchecked Exceptions

- Occur at **runtime**
- Not checked by compiler
- Subclasses of `RuntimeException`

Examples:

- `NullPointerException`
- `ArithmeticException`

- `ArrayIndexOutOfBoundsException`

```
inta=10 /0;// ArithmeticException
```

5. Core Keywords in Exception Handling

Keyword	Purpose
<code>try</code>	Wraps risky code
<code>catch</code>	Handles exceptions
<code>finally</code>	Executes whether exception occurs or not
<code>throw</code>	Explicitly throws an exception
<code>throws</code>	Declares exceptions to caller

6. try-catch Block

```
try {  
    inta=10 /0;  
}catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

How it works

- JVM monitors the `try` block
- If an exception occurs, control transfers to the matching `catch`

7. Multiple catch Blocks

```
try {  
    int[] arr =newint[5];
```

```
arr[10] = 50;
}catch (ArithmeticException e) {
    System.out.println("Arithmetic error");
}catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array index error");
}
```

Rule: Catch specific exceptions before general ones

8. finally Block

- Always executes
- Used for **resource cleanup** (closing files, database connections)

```
try {
    intx=5 /1;
}catch (Exception e) {
    System.out.println("Error occurred");
}finally {
    System.out.println("This always executes");
}
```

9. throw Keyword

Used to **explicitly throw** an exception.

```
throw new ArithmeticException("Invalid operation");
```

Used when you want to create **custom error conditions**.

10. throws Keyword

Used in method declaration to pass exception handling responsibility to the caller.

```
void readFile() throws IOException {  
    FileReader fr = new FileReader("file.txt");  
}
```

11. Custom (User-Defined) Exceptions

You can create your own exception by extending `Exception` or `RuntimeException`.

```
class InvalidAgeException extends Exception {  
    InvalidAgeException(String msg) {  
        super(msg);  
    }  
}
```

Usage:

```
if (age < 18) {  
    throw new InvalidAgeException("Age must be 18 or above");  
}
```

12. try-with-resources (Java 7+)

Automatically closes resources that implement `AutoCloseable`.

```
try (FileReader fr = new FileReader("file.txt")) {  
    // use file  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

```
}
```

✓ No need for `finally` to close resources

File Handling in Java

1. What Is File Handling?

File handling is the process of **managing files and directories** using Java programs, including:

- Creating files
- Writing data to files
- Reading data from files
- Appending data
- Deleting files

2. Java File Handling Packages

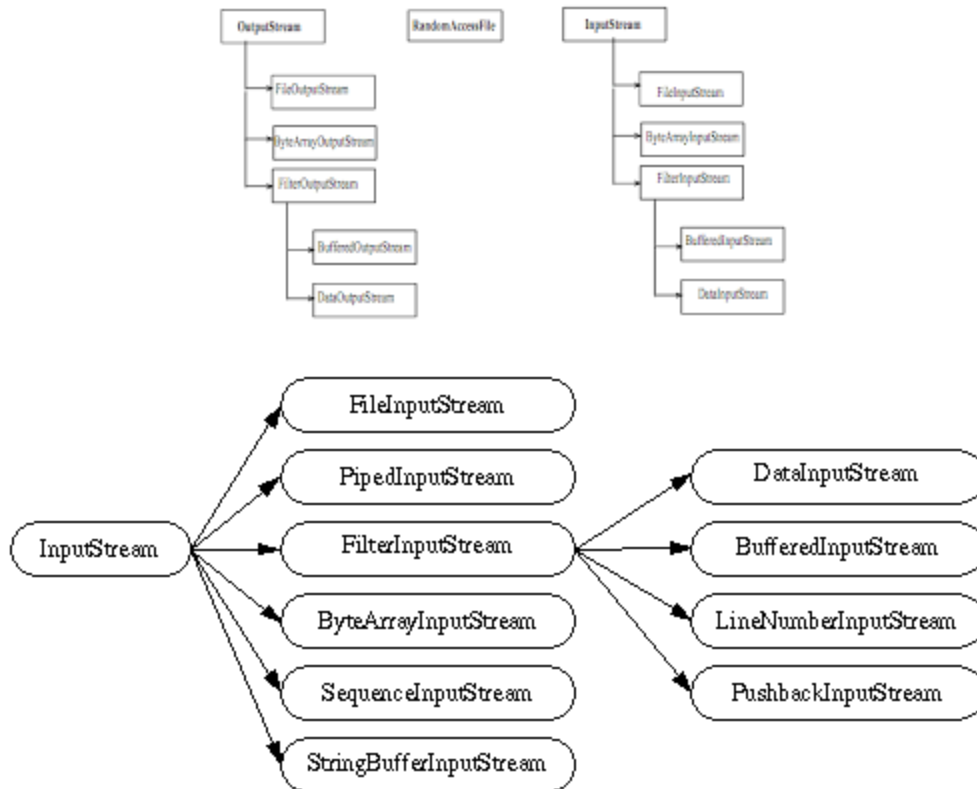
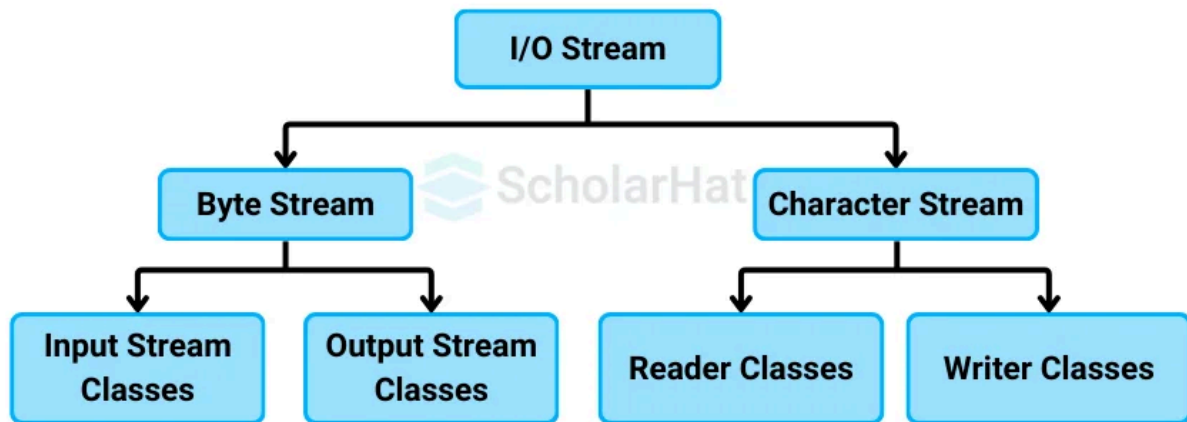
Java provides file-handling support mainly through:

Package	Purpose
<code>java.io</code>	Traditional file I/O (streams, readers, writers)
<code>java.nio.file</code>	Modern file I/O (faster, scalable, introduced in Java 7)

3. File Handling Architecture



Stream in Java



Two main concepts

1. **Stream** – Sequence of data
2. **File** – Storage location on disk

4. Streams in Java

4.1 Types of Streams

Stream Type	Description
Byte Stream	Handles binary data (images, audio)
Character Stream	Handles text data

4.2 Byte Streams

- Base classes:

- `InputStream`
- `OutputStream`

Examples:

- `FileInputStream`
- `FileOutputStream`

```
FileInputStream fis = new FileInputStream("data.bin");
int i;
while ((i = fis.read()) != -1) {
    System.out.print((char)i);
}
fis.close();
```

4.3 Character Streams

- Base classes:

- `Reader`
- `Writer`

Examples:

- `FileReader`

- `FileWriter`

```
FileReader fr = new FileReader("data.txt");
int i;
while ((i = fr.read()) != -1) {
    System.out.print((char)i);
}
fr.close();
```

✓ Best for text files

5. File Class (`java.io.File`)

Used to represent **files and directories**.

```
File file = new File("sample.txt");
```

Common methods

Method	Description
<code>exists()</code>	Checks if file exists
<code>createNewFile()</code>	Creates a new file
<code>delete()</code>	Deletes file
<code>getName()</code>	Returns file name
<code>getAbsolutePath()</code>	Returns full path
<code>length()</code>	Returns file size

6. Writing Data to a File

6.1 Using FileWriter

```
FileWriterfw=newFileWriter("data.txt");  
fw.write("Hello Java File Handling");  
fw.close();
```

6.2 Appending Data

```
FileWriterfw=newFileWriter("data.txt",true);  
fw.write("\nAppending text");  
fw.close();
```

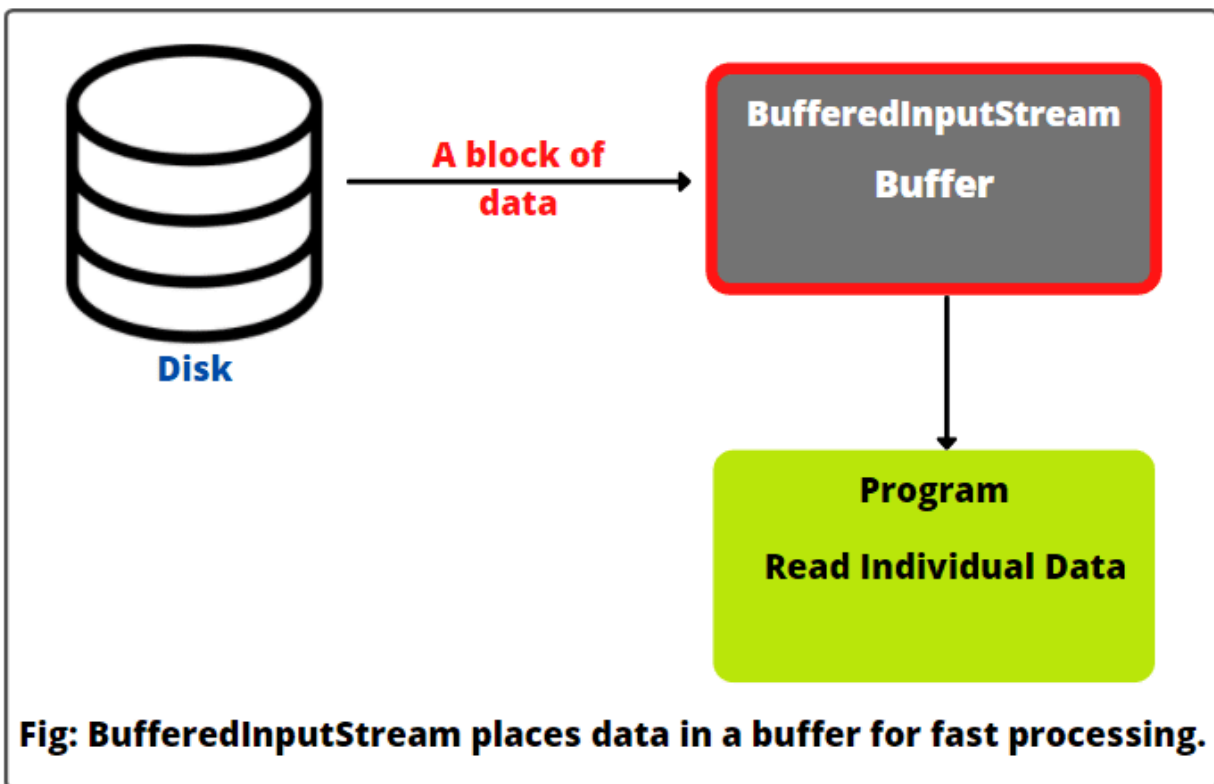
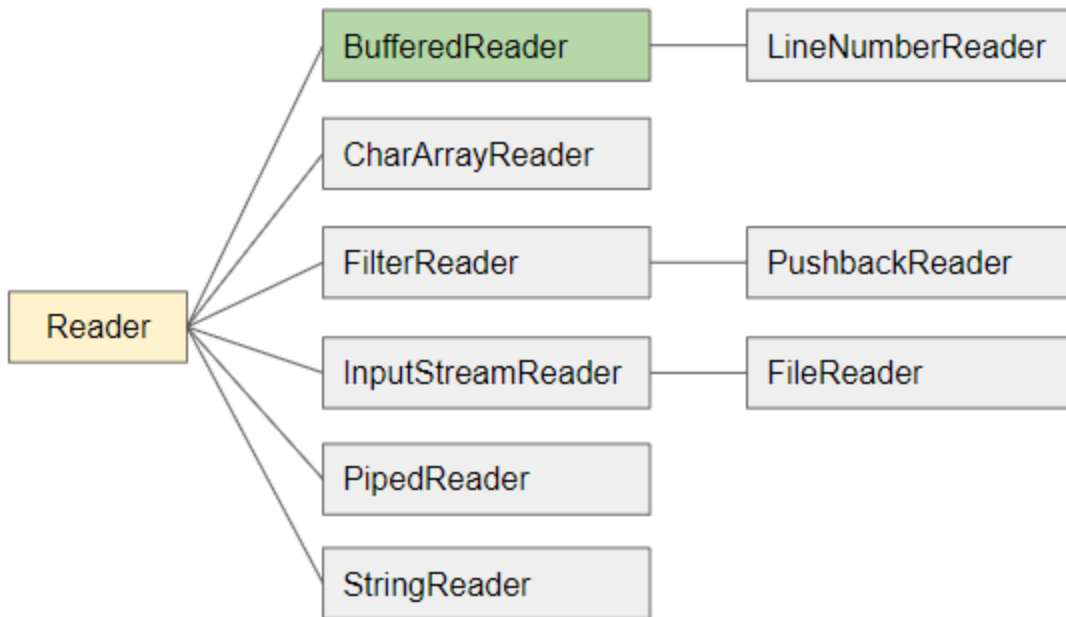
7. Reading Data from a File

Using BufferedReader (Efficient)

```
BufferedReaderbr=newBufferedReader(newFileReader("data.txt"));  
String line;  
while ((line = br.readLine()) !=null) {  
    System.out.println(line);  
}  
br.close();
```

✓ Faster due to buffering

8. Buffered Streams



Why use buffering?

- Reduces disk access

- Improves performance

Examples:

- `BufferedInputStream`
- `BufferedOutputStream`
- `BufferedReader`
- `BufferedWriter`

9. try-with-resources (Best Practice)

Automatically closes files.

```
try (FileReader fr = new FileReader("data.txt")) {  
    int i;  
    while ((i = fr.read()) != -1) {  
        System.out.print((char)i);  
    }  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

✓ No need for `finally`